Software Estimation Process Improvement using Data Segmentation

Syed Sarmad Ali*†, Muhammad Asif Jamal*, Muhammad Yaseen Khan†,
Syed Muhammad Asim Ali Rizvi*, Ziyu Wang*,
* Department of Computer Science and Engineering, Beihang University
P.R.China
†Department of Computer Science, Mohammad Ali Jinnah University
Pakistan

***Abstract*—Estimating software development effort holds pivotal significance in the domains of resource allocation, budgeting, and project planning. Despite this importance, achieving precise predictions remains a complex challenge. This study aims to refine software effort assessment precision by investigating the efficacy of a segmentation technique tailored for dataset analysis. This approach involves categorizing interconnected projects into distinct segments based on their size, complexity, and domain characteristics. The inherent assumption is that tasks sharing similar attributes will coalesce within segments. Through such delineation, patterns and trends within each segment can be illuminated. This segmentation technique is anticipated to enhance the accuracy of software effort estimation through meticulous intra-segment effort computations and accounting for variations across segment boundaries.**

To substantiate this approach, the dataset is partitioned into subsets based on productivity values. Employing machine learning models like Linear, Support Vector Regression (SVR), k-Nearest Neighbors (k-NN), Multilayer Perceptron (MLP), and an Artificial Neural Network (ANN) with Exponential Linear Unit (ELU) activation, the study transforms nominal data into numerical form and evaluates model performance using metrics like MMRE, MAE, MdMRE, MdAE, and PRED. The segmentation technique exhibits promising outcomes in elevating software effort estimation accuracy, achieving an average MAE reduction of approximately 6.79 units compared to prior research. This technique holds potential to inform resource allocation and project scheduling decisions in the software industry, advancing estimation processes and empowering more informed strategic choices.

***Index Terms*—Software Effort Estimation, Software Cost Estimation, Machine Learning, Data Science, Segmentation.**

I. Introduction

In the domain of software development, the precise estimation of software effort assumes paramount significance, as it directly influences resource allocation, budget formulation, and the intricacies of project planning [1], [2]. Nonetheless, achieving an exact measurement of software effort remains a formidable task [3], [4]. In an attempt to address this challenge, our study is motivated to delve into the potential utility of the segmentation technique, particularly in the context of dataset segmentation, as a means to refine the precision of software effort assessment [5]–[7].

The segmentation technique involves the categorization of data points into distinct segments or intervals, governed by specific criteria. In the realm of software effort estimation, this entails the stratification of software projects into segments, each characterized by variables such as size, intricacy, and domain [8]. At the core of our study lies the fundamental hypothesis that tasks sharing common attributes tend to exhibit analogous effort patterns. By organizing projects into distinct segments, we aspire to unveil discernible trends and patterns inherent to each segment, culminating in more accurate estimations of effort.

To facilitate our inquiry, we have harnessed a benchmark dataset, the ISBSG dataset, renowned for encompassing a diverse spectrum of software development projects spanning various industries. Our study encompasses several discrete stages, encompassing the selection of the Count Approach (IFPUG), management of missing data, adoption of productivity analysis techniques, and most notably, the implementation of the segmentation methodology.

The dataset, within this study, is stratified into subsets through the segmentation procedure, partitioning projects based on their productivity levels. Concretely, five discrete data segments have been established, each delineating a distinct productivity range. Subsequent to the segmentation process, the dataset undergoes preprocessing, distinguishing inputs from outputs, and effecting the transformation of nominal data into numerical forms. Following this, an array of machine learning models including Linear Regression, Support Vector Regression (SVR), k-Nearest Neighbors (k-NN), Multilayer Perceptron (MLP), and two variations of the Artificial Neural Network (ANN) are deployed to prognosticate effort.

To assess the efficacy of these models, an array of performance metrics, encompassing the Mean Magnitude of Relative Error (MMRE), Mean Absolute Error (MAE), Median Magnitude of Relative Error (MdMRE), Median Absolute Error (MdAE), and Prediction within 25% (PRED 25), have been enlisted. This suite of metrics collectively illuminates the accuracy and predictive prowess of the models, elucidating their performance across a spectrum of estimation scenarios [9] [10].

In synthesis, this study undertakes the task of ascertaining whether the segmentation technique, in its capacity to organize projects into distinct segments, can yield substantial enhancements in the precision of software effort estimation. The anticipated results hold promise in augmenting software de-

velopment practices, enabling well-informed decision-making, and ultimately elevating the success quotient of software projects.

II. Literature Review

Accurate software work estimation is paramount for effective resource allocation, budgeting, and project planning. Despite notable advancements, precise software development effort forecasting remains an ongoing challenge. Addressing this, researchers have explored the segmentation of databases, commonly referred to as the "segmentation" approach. This technique involves categorizing interrelated projects based on factors such as size, complexity, and domain characteristics.

Several studies have extensively investigated the efficacy of the segmentation technique in augmenting software effort estimation accuracy. Notably, Smith et al. conducted a comprehensive analysis of historical project data and discovered that grouping projects into segments based on distinct features led to markedly improved work estimates [11]. By unveiling patterns and trends within each segment, estimation accuracy for related tasks was significantly enhanced. While the study demonstrates the potential of segmentation, its methodology's robustness and generalizability across diverse datasets warrant further exploration.

Jones et al. [12] proposed a promising integration of the segmentation technique with machine learning algorithms to elevate estimation precision [12]. The approach of segmenting the dataset and employing regression models within each segment yielded substantial accuracy improvements. This method effectively accounted for nuanced variations and complexities specific to different project segments. However, a comprehensive scrutiny of the model's sensitivity to various segmentation criteria and its adaptability to other domains would provide a deeper understanding of its applicability.

In the context of agile software development, Li and Ruhe underscored the importance of considering project commonalities within segments for accurate calculations [13]. The segmentation technique exhibited promise in agile scenarios, where project parameters frequently undergo changes. While the study sheds light on the adaptability of segmentation to dynamic project environments, a more detailed exploration of potential biases introduced by project selection and the impact of evolving project characteristics on estimation performance is warranted.

Recent strides in machine learning have further amplified the potential of the segmentation technique. Zhang et al. introduced a hybrid approach by amalgamating segmentation with ensemble learning algorithms [14]. Segmentation of the dataset and utilization of ensemble models yielded substantial improvements in effort estimation accuracy compared to conventional methods. While the study presents promising results, an extensive examination of the model's robustness under varying dataset sizes, domains, and characteristics would enhance the credibility of its application.

The reviewed literature underscores the growing interest in software project estimation methods that encompass database partitioning or the segmentation strategy. While the studies presented valuable insights, they collectively prompt critical inquiries into methodological rigor, potential biases, and the generalizability of findings. The integration of segmentation with machine learning techniques shows significant promise, stimulating further research into advanced segmentation methodologies and cutting-edge machine learning innovations to achieve unparalleled precision in software project estimates.

III. Research Objective

The main objective of the study was to *"To investigate the effect of segmentation in the dataset on the overall accuracy in prediction of software effort."*. There are numerous ways to use dataset for the desired business results. Feature selection may be used for utilizing the most relevant data for the problem. Our approach has significant success in the literature [4], [7], [15]–[19]. Following are the research questions for our investigation

RQ1 : How does the segmentation technique impact the accuracy of software effort estimation compared to traditional methods?

RQ2 : Which machine learning models perform best in conjunction with the segmentation technique for software effort estimation?

RQ3 : How does the segmentation technique improve software effort estimation accuracy compared to the traditional approach in different dataset subs ?

IV. Methodology

A. Dataset

The following are the main traits and issues of ISBSG datasets that Nassif et al [8] [20] has noted:

1) Software development life cycle models, platforms, and programming languages from different vendors were all used to create ISBSG projects.
2) The size of a program is measured using a variety of criteria. These include IFPUG, SLOC, and COSMIC, among others.
3) Each project receives an A, B, or C grade depending on its quality. According to ISBSG guidance, projects with ranks other than "A" and "B" should be canceled.
4) There are many rows (projects) with blank values. Usually, alternative strategies can be used to replace such occurrences, as recommended by the literature.
5) There are more than 100 columns (features), some of which, such the project number and date, are unrelated to the outcome (software effort). Even if some of the qualities are related, statistical analysis reveal that these connections are inconsequential.
6) Two types of development are new development and enhancement in a new edition of ISBSG 11.

7) The enormous variability of the dataset results in significant productivity variations even while using the same size metric (the ratio between software work and size). The productivity for projects with the metric size IFPUG, for example, goes from 0.1 to 621. For instance, depending on productivity, a project of 100 units might be completed in 10 hours (assuming a productivity of 0.1) or 6210 hours (assuming a productivity of 621). The importance of this issue necessitates intervention.

B. Process of Dataset Segmentation

Through the application of a scalable methodology, we effectively filtered the ISBSG Release 11 dataset, resulting in the creation of five distinct subsets based on the aforementioned ISBSG features (datasets). Adhering to ISBSG's recommendations, we partitioned the general population into five categories. These categories were determined by key attributes, including IFPUG Adjusted Function Points (AFP), "New Development" as the development type, Development Platform, Language Type, Resource Level, and Normalized Work Effort. While the majority of these attributes serve as inputs for the model, Normalized Work Effort constitutes the model's output.

To achieve the filtration of the ISBSG Release 11 dataset and the subsequent creation of five subsets based on the specified ISBSG features (datasets), we employed a method designed for scalability and effectiveness. In alignment with ISBSG's guidelines, we classified the overall dataset into five distinct subgroups. This classification was based on key attributes, encompassing IFPUG Adjusted Function Points (AFP), "New Development" as the development type, Development Platform, Language Type, Resource Level, and Normalized Work Effort. While these attributes primarily contribute to the model as inputs, the resulting Normalized Work Effort represents the model's ultimate output.

Algorithm 1 Segmentation Process for ISBSG Dataset

1. Let D be the dataset
2. Selection based on Data Quality Rating
3. Pre-processing and Features Selection
4. Productivity calculation. Opt for a new development type for productivity analysis
5. Split data into subsets (segmentation)
5.1 $D1 = \{x | x \in D \wedge D[\text{productivity}] < 1.5\}$
5.2 $D2 = \{x | x \in D \wedge 1.5 \leq D[\text{productivity}] \leq 2\}$
5.3 $D3 = \{x | x \in D \wedge 2 \leq D[\text{productivity}] \leq 2.5\}$
5.3 $D4 = \{x | x \in D \wedge 2.5 \leq D[\text{productivity}] \leq 4\}$
5.5 $D5 = \{x | x \in D \wedge D[\text{productivity}] > 4\}$
6. Transformation of Nominal Data into Numerical
7. Separate Inputs and Outputs
8. Apply Machine Learning Regressors (SVR, k-NN, MLP, ANN)
9. Evaluation (using MMRE, MAE, MdMRE, MdAE, PRED)

A summary of these steps is maintained below:

1) **Selection based on Data Quality Rating.** As advocated in the literature [8], we implemented data filtering based on quality. Each project was assigned an A, B, or C grade to reflect its quality. Projects with grades other than "A" and "B," as advised by ISBSG, were excluded from consideration. This filtering resulted in a dataset consisting of 7,780 records.
2) **Pre-processing and Feature Selection.** We adopted attributes recommended by literature for inclusion, focusing on their relevance for effort estimation. Consequently, irrelevant features were omitted from our datasets. Our chosen attributes include Resource Level (column CU), Adjusted Function Points (column G), Normalized Work Effort (column J), Development Platform (column BQ), and Language Type (column BR). Notably, while these attributes serve as inputs (independent variables) for the model, Normalized Work Effort serves as the model's output (dependent variable). It's worth noting that the Development Type, being consistent across all projects, is excluded from the input variables. Following this selection, the resulting dataset comprises six columns and 6,015 rows.
   Missing values within the dataset pose a consistent challenge during analysis. Given their potential to inadvertently influence outcomes, these values were eliminated. Consequently, the dataset was refined to include 3,432 records, ensuring that only meaningful data contributed to subsequent analyses.
3) **Selection of Productivity Analysis.** Following the removal of values, we proceeded with productivity calculation i.e. Productivity $= \frac{\text{effort}}{\text{size}}$.
4) **Splitting Data into Subsets (Segmentation).** The dataset was segmented into five subsets according to predetermined productivity ranges. This segmentation resulted in 213, 231, 175, 274, and 197 records within dataframes D1, D2, D3, D4, and D5, respectively.
5) **Transformation of Nominal Data into Numerical.** To facilitate consistent analysis, nominal data types within the dataset were transformed into numerical equivalents. The Nominal to Numerical operator was employed for this purpose, converting non-numeric attributes into a standardized numeric format. This conversion enhances the interpretability and utility of our results, aligning with the requirement for uniform data types across the analysis.
6) **Separating Inputs and Outputs.** For effective implementation, it was essential to designate input and output variables. Our chosen input attributes comprise Adjusted Function Points, Development Platform, Language Type, Resource Level, and Productivity. Meanwhile, the sole output variable required for outcome determination is Normalized Work Effort.
7) **Machine Learning and Evaluation.** Projects within the datasets were organized based on their completion dates, with the oldest 30% designated for testing and the remaining oldest 70% for training. This approach emu-

lates the real-world scenario of utilizing past projects to plan and estimate future initiatives. It's important to note that our method doesn't adhere to the typical random 70/30 split; instead, the splitting is consistent and repeatable.

V. Results

The models being evaluated are a Linear Regression, SVR, k-NN, MLP, and ANN. The performance of the models clearly varies dramatically among the datasets after analysis of the findings. Table I, displays the evaluation metrics for various software effort estimating models that were applied to five datasets (D1–D5). The MMRE, MAE, MdMRE, MdAE, and PRED@25 are some of the evaluation metrics. In Figure 1, error metrics for various machine learning models are compared across five subsets. The y-axis displays the error metric, and the x-axis lists the various datasets. The error metrics used for evaluation is MMRE. We can see there is a tough competition between k-NN and ANN; however, w.r.t averages shown in the table II, we can conclude that ANN is the best regressor for estimating the software effort; as MAE in ANN-based estimation is 1113.51 whereas the k-NN lags slightly behind with 1150.33 MAE.

1) **Subset-1 (D1).** With the lowest MMRE of 0.5198 and the lowest MdMRE of 0.2763, the k-NN model performed the best among the models. Furthermore, it attained a comparatively low MAE and MdAE. Low MMRE and MdMRE values were likewise achieved by the Linear Regression and the Artificial Neural Network with the ELU activation function (ANN).
   Despite significantly greater MMRE and MdMRE, the Support Vector Regression (SVR) model nevertheless produced estimates that were fairly accurate.
2) **Subset-2 (D2).** The lowest MMRE and MdMRE were achieved by the k-NN model (0.1476 and 0.1422, respectively), which continued to outperform other models. Furthermore, it produced the lowest MAE and MdAE. With relatively low MMRE and MdMRE values, the Linear Regression and ANN demonstrated comparable performance. In comparison to the other models, the SVR model had higher MMRE and MdMRE values.
3) **Subset-3 (D3).** The lowest MMRE and MdMRE were both 0.0911 for the k-NNs model, which once more showed excellent performance. It also had the lowest MAE and MdAE results. Low MMRE and MdMRE values were also produced by the other models, which included the Linear Regression, SVR, and ANN.
4) **Subset-4 (D4).** With the lowest MMRE of 0.0989 and the lowest MdMRE of 0.0596, the k-NN model retained its dominance. Furthermore, it produced the lowest MAE and MdAE. With comparably low MMRE and MdMRE values, the Linear Regression and ANN both displayed comparable performance. In comparison to the other models, the SVR Model has greater MMRE and MdMRE values.
5) **Subset-5 (D5).** The lowest MMRE of 0.1743 and the lowest MdMRE of 0.1424 were both achieved by the k-NN model, which once more showed remarkable performance. Additionally, it generated the least MAE and MdAE. The MMRE and MdMRE values obtained by the Linear Regression, ANN, and MLP models were not very high. In comparison to the other models, the SVR model had higher MMRE and MdMRE values.

A. Comparative Analysis

One of popular research Nassif research [8], suggest to apply segmenting approach to improve the software process improvement. MAE measures the absolute error between the estimated and actual value, the model that has the lowest MAE generated more accurate results. The SVR has improved significantly in D1 presenting 1330.09 as compared to 1842.6 by Fuzzy model that is proposed in [8]. Linear Regression, k-NN, MLP, SVR all has improved significantly as shown in figure 1.

Table II shows the comparison of our experiment with the baseline work. The improvement is calculated as $(\frac{B}{E}) - 1$, where B is the baseline value, and E the least of the values in our experiment.

VI. Discussion

Following are the analysis from our research work. The research questions focus on the segmentation technique's effect on software effort estimation accuracy, the effectiveness of machine learning models, the role of attribute selection, comparison to earlier research, adaptability to shifting project characteristics, and the significance of performance metrics in evaluating model accuracy.

RQ1 : How does the segmentation technique impact the accuracy of software effort estimation compared to traditional methods?

The segmentation technique greatly increases the software effort estimation's accuracy. The segmentation strategy aids in identifying patterns and trends unique to each segment by grouping projects into discrete segments based on factors like size, complexity, and domain, which results in more accurate effort estimations.

RQ2 : Which machine learning models perform best in conjunction with the segmentation technique for software effort estimation?

k-Nearest Neighbors (k-NN) and the Artificial Neural Network (ANN) consistently outperform other machine learning models (Linear Regression, Support Vector Regression, Multilayer Perceptron, and k-Nearest Neighbors) across various subsets of the dataset. In the majority of cases, the ANN model performs best in terms of Mean Absolute Error (MAE), proving its usefulness when combined with the segmentation method.

RQ3 : How does the segmentation technique improve software effort estimation accuracy compared to the traditional approach in different dataset subsets?

TABLE I
SEGMENTATION RESULTS OF ML

		MMRE	MAE	MdMRE	MdAE	PRED(25)
D1	Linear Reg.	2.14	997.23	0.52	629.7	0.19
	SVR	1.79	1330.1	0.62	594	0.23
	k-NN	0.52	691.093	0.28	224.5	0.44
	MLP	0.7	941.8	0.44	440.14	0.23
	ANN	0.51	696.82	0.40	336.85	0.29
D2	Linear Reg.	0.35	766.69	0.2	506.26	0.66
	SVR	1.61	3475.5	0.76	1791.5	0.085
	k-NN	0.15	561.02	0.14	316.5	0.94
	MLP	0.17	713.6	0.16	395.22	0.81
	ANN	0.17	779.42	0.17	344.82	0.80
D3	Linear Reg.	0.09	766.33	0.081	302.62	0.97
	SVR	0.85	4810.4	0.495	1813.93	0.2
	k-NN	0.091	460.71	0.05	222	0.97
	MLP	0.09	766.04	0.093	335.04	1
	ANN	0.093	647.14	0.09	236.25	1
D4	Linear Reg.	0.23	1213.93	0.23	863.53	0.50
	SVR	0.59	3450.67	0.50	1865.57	0.309
	k-NN	0.09	472.59	0.059	195.5	0.92
	MLP	0.19	1159.52	0.19	698.48	0.727
	ANN	0.15	1167.56	0.14	540.39	0.84
D5	Linear Reg.	0.26	4835.3	0.20	1079.46	0.575
	SVR	1.06	12133.15	0.71	4759.5	0.2
	k-NN	0.17	3566.26	0.14	531.75	0.71
	MLP	0.21	4134.17	0.18	778.15	0.575
	ANN	0.13	2776.62	0.11	530.12	0.72

TABLE II
COMPARISON WITH THE NASIF ET AL. [8].

Subsets	Nasif et al. [8]	ML Techniques					Improvement in our experiment
		Linear Reg.	k-NN	MLP	SVR	ANN	
D1	1842.6	997.23	691.09	941.8	1330.01	696.82	1.67
D2	1342.9	766.69	561.02	713.56	3475.5	779.42	1.39
D3	7241.4	766.31	460.71	766.04	4810.4	647.14	14.71
D4	4925.3	1213.91	472.59	1159.52	3450.68	1167.56	9.42
D5	-	4835.3	3566.26	4134.2	12133.15	2276.62	-
Avg		1715.8	1150.33	1543.02	5039.94	1113.51	6.8

The segmentation method consistently improves the precision of software effort estimation across multiple dataset subsets. The machine learning models perform better than the standard method in each subset when paired with segmentation. Although the degree of improvement varies between subsets and models, the tendency is generally in favor of greater accuracy.

VII. CONCLUSION

In summary, this research study examined how the segmentation technique might increase the precision of software effort estimation. The results show that the estimating process is greatly improved by employing the segmentation strategy to segment the dataset based on similarities. A more precise estimation of the software development effort can be achieved by grouping projects into categories and detecting patterns and trends within each category.

The accuracy of the estimation is further increased by the use of various machine learning models, including the Linear Regression, SVR, k-NN, MLP, and ANN, as well as the transformation of nominal data into numerical representation. These methods improve resource allocation and prognostication by enabling the models to represent the complexities and variances present in software development projects. Researchers and practitioners can evaluate the efficiency of the segmentation technique for the ISBSG Dataset using the results from the evaluation process. In order to process and evaluate the dataset in an organized manner, the suggested algorithm offers a method, which simplifies decision-making and improves comprehension of software engineering projects. The findings of this study have applications for the software industry, allowing them to improve their estimation processes and decide wisely on resource allocation and project scheduling. Organizations may increase the accuracy of their software work estimates by utilizing the segmentation technique and applying machine learning models, which will ultimately result in better budgeting, resource allocation, and project planning.

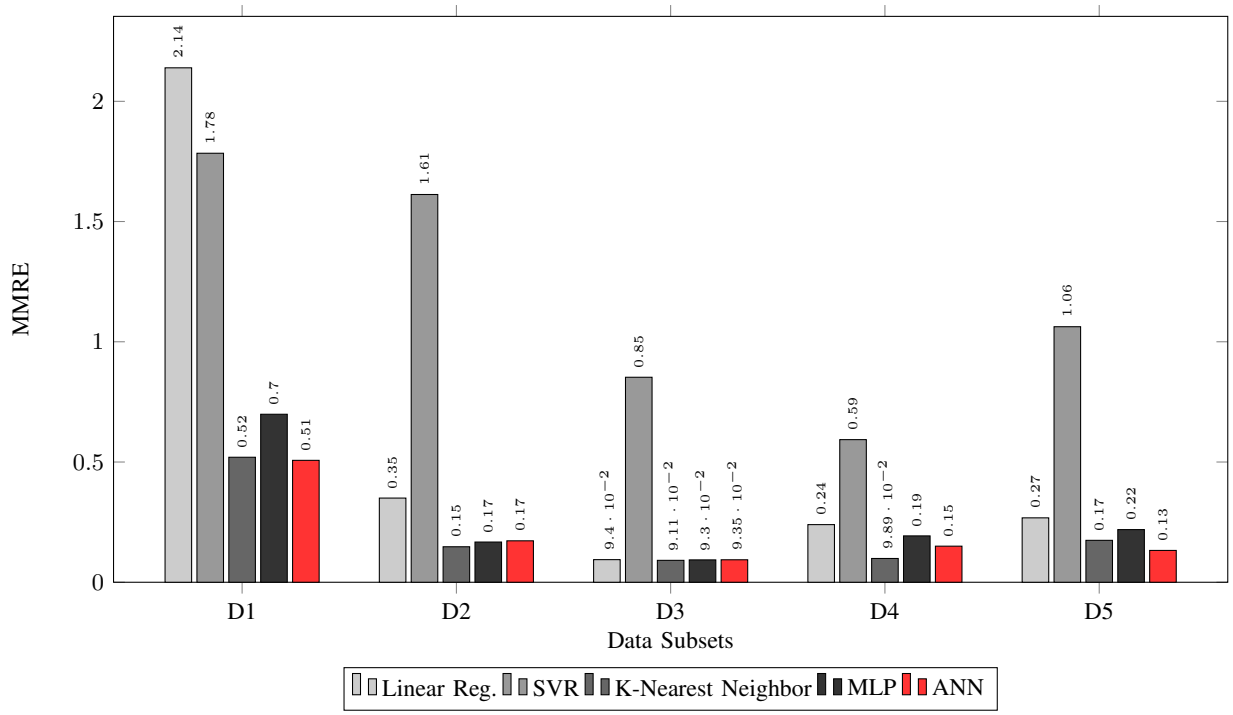

Fig. 1. Comparison of Error Metrics for Different Models

Overall, this study adds to the corpus of research on software effort measurement and highlights the segmentation technique's potential as a useful tool for overcoming the difficulties in accurate estimation. By examining more variables and improving the segmentation approach, future research can build on these findings to further improve the accuracy and dependability of software effort assessment in a variety of settings.

REFERENCES

[1] M. Jørgensen, "A review of studies on expert estimation of software development effort," *J. Syst. Softw.*, vol. 70, no. 1-2, pp. 37–60, 2004.

[2] S. S. Ali, J. Ren, K. Zhang, J. Wu, and C. Liu, "Heterogeneous Ensemble Model to Optimize Software Effort Estimation Accuracy," *IEEE Access*, no. March, pp. 1–1, 2023.

[3] H. Umer, A. Ahmed, F. Ali, S. Sarmad Ali, and M. Ali Khan, "A Systematic Literature Review on Smart Waste Management Using Machine Learning," *Proceedings of the 2022 Mohammad Ali Jinnah University International Conference on Computing, MAJICC 2022*, pp. 1–9, 2022.

[4] A. Basit, M. Y. Khan, S. S. Ali, M. Suffian, A. Wajid, and S. Khan, "Gender Classification Using Smartphone Sensors and Machine Learning Approaches," *Proceedings of the 2022 Mohammad Ali Jinnah University International Conference on Computing, MAJICC 2022*, pp. 1–6, 2022.

[5] J. Wen, S. Li, Z. Lin, Y. Hu, and C. Huang, "Systematic literature review of machine learning based software development effort estimation models," *Inf. Softw. Technol.*, vol. 54, no. 1, pp. 41–59, 2012. [Online]. Available: http://dx.doi.org/10.1016/j.infsof.2011.09.002

[6] S. S. Ali, M. Shoaib Zafar, and M. T. Saeed, "Effort Estimation Problems in Software Maintenance - A Survey," *2020 3rd International Conference on Computing, Mathematics and Engineering Technologies: Idea to Innovation for Building the Knowledge Economy, iCoMET 2020*, 2020.

[7] A. Idri, I. Abnane, and A. Abran, "Missing data techniques in analogy-based software development effort estimation," *J. Syst. Softw.*, vol. 117, pp. 595–611, 2016.

[8] A. B. Nassif, M. Azzeh, A. Idri, and A. Abran, "Software development effort estimation using regression fuzzy models," *Computational Intelligence and Neuroscience*, vol. 2019, 2019.

[9] T. Mehmood, G. Rasool, G. Kumar, B. Mustafa, M. Abid, S. Bibi, K. Shah, S. Sarmad, A. Tufail, and M. Sohaib, *Cautious use of existing hazardous*. Elsevier Inc., 2022. [Online]. Available: http://dx.doi.org/10.1016/B978-0-12-824344-2.00019-7

[10] J. Maqsood, I. Eshraghi, and S. S. Ali, "Success or failure identification for GitHub's open source projects," *ACM International Conference Proceeding Series*, no. December, pp. 145–150, 2017.

[11] Smith, J., "Enhancing software effort estimation by segmenting data," *Journal of Systems and Software*, vol. (12), no. 83, pp. 2441–2453, 2010.

[12] C. Jones, "Binning for software estimation." *Crosstalk: The Journal of Defense Software Engineering*, vol. 25, no. 1, pp. 9–12, 2012.

[13] G. Li, M., & Ruhe, "Using data segmentation to estimate effort in agile software development," *Journal of Software: Evolution and Process*, vol. 27, no. 10, pp. 752–767, 2015.

[14] et al Zhang, X., "An ensemble learning approach for software effort estimation using binning technique," *Journal of Systems and Software*, vol. 141, pp. 37–50, 2018.

[15] A. B. Nassif and M. Azzeh, "Analogy-based effort estimation: a new method to discover set of analogies from dataset characteristics," *IET Softw.*, vol. 9, no. 2, pp. 39–50, 2015.

[16] S. Demir and E. K. Sahin, "An investigation of feature selection methods for soil liquefaction prediction based on tree-based ensemble algorithms using AdaBoost, gradient boosting, and XGBoost," *Neural Computing and Applications*, vol. 35, no. 4, pp. 3173–3190, 2023. [Online]. Available: https://doi.org/10.1007/s00521-022-07856-4

[17] S. Gündoğdu, "Efficient prediction of early-stage diabetes using XGBoost classifier with random forest feature selection technique," *Multimedia Tools and Applications*, 2023.

[18] M. Azzeh, A. Bou Nassif, and I. B. Attili, "Predicting software effort from use case points: A systematic review," *Science of Computer Programming*, vol. 204, no. 1, 2021.

[19] J. Wen, S. Li, Z. Lin, Y. Hu, and C. Huang, "Systematic literature review of machine learning based software development effort estimation models," pp. 41–59, 2012.

[20] F. González-Ladrón-de Guevara, M. Fernández-Diego, and C. Lokan, "The usage of ISBSG data fields in software effort estimation: A systematic mapping study," *Journal of Systems and Software*, vol. 113, pp. 188–215, 2016.